

Build Your Own Mini Docker

Understanding containers from scratch

Sayan

Paradox 2026 | IIT Madras | 11 June

What is a container,
in syscalls?

Before we touch any of that

- Entry ticket: a **green pre-flight** on your laptop.
- Get the code:

```
git clone https://github.com/sayan-iitm/minidocker
cd minidocker
```

- Fell behind at any point? `bash demo/cpN/run.sh`.

Namespaces

A namespace is an **isolated view** of one global thing.

mount	the filesystem tree
UTS	the hostname
PID	process IDs
net	interfaces, routes, ports
user	uid / gid — root inside $\neq$ root outside
IPC	shared memory, message queues
cgroup	the cgroup tree you can see

One command runs the lot

```
unshare --uts --pid --mount --user --net --ipc --cgroup ...
```

- `lsns` — the namespaces you're already in.
- The next *~100 lines of bash* just glue these together.

100 lines of bash

Where we're going

- | | |
|--------------------------------------|-------------|
| 1. CP1 — hostname | UTS |
| 2. CP2 — our own process list | PID + mount |
| 3. CP3 — drop <code>sudo</code> | user |
| 4. CP4 — our own root filesystem | pivot_root |
| 5. CP5 — a limit the kernel enforces | cgroups |

then tie it together → `mini-docker.sh`

Lost? `bash demo/cpN/run.sh` catches you up.

Ctrl-C did nothing. Why?

- Inside the container, our shell is `PID 1`.
- The kernel `drops` `SIGINT`/`SIGTERM` to PID 1 unless it installs a handler.
- Real runtimes ship a tiny init (`tiny`) that becomes PID 1, forwards signals, reaps zombies.

That is what `docker run --init` is for.

What Docker adds on top

- A tarball of **layers** + metadata (what to run, env, ports).
- Our `rootfs/` was step one of this, by hand.

- Stack read-only layers; add one thin **writable** layer on top.
- Share the base across containers $\Rightarrow$ fast, cheap.

- A recipe. Each instruction → one cached layer.
- Reproducible images instead of "works on my machine".

- `push` / `pull` images by name + tag.
- Docker Hub, GHCR, your own — just content-addressed blobs.

- bridge + veth pairs + iptables → real container networking.
- seccomp, capabilities, AppArmor → shrink what the container can do.
- `--init` — the PID-1 fix we just met.

The bridge: goboxd

Same problems — we hit them naively

Our bash	goboxd spec hole
<code>pivot_root + umount</code>	path traversal
run a process, not <code>bash -c "..."</code>	shell-style commands
<code>user ns + --map-root-user</code> (not handled)	UID collisions under load
<code>trap</code> cleanup	unbounded output — <code>rlimit</code> stale jail directories

nsjail = our script, hardened

- Everything we wrote, in ~6000 lines of C.
- **plus** seccomp, capability dropping, a config language.
- **goboxd**: wrap it in a Go service that runs untrusted code.

- Catch the `goboxd` finals — tomorrow. You've got the mental model now.
- Repo & slides: `github.com/sayan-iitm/minidocker`
- Stuck / curious: GitHub Discussions on the repo.

Questions?